

20200517面试题

阅读下面代码，我们只考虑浏览器环境下的输出结果，写出它们结果打印的先后顺序，并分析出原因，小伙伴们，加油哦！

```
1 console.log("AAAA");
2 setTimeout(() => console.log("BBBB"), 1000);
3 const start = new Date();
4 while (new Date() - start < 3000) {}
5 console.log("CCCC");
6 setTimeout(() => console.log("DDDD"), 0);
7 new Promise((resolve, reject) => {
8   console.log("EEEE");
9   foo.bar(100);
10 })
11 .then(() => console.log("FFFF"))
12 .then(() => console.log("GGGG"))
13 .catch(() => console.log("HHHH"));
14 console.log("IIII");
```

答案：

浏览器下 输出结果的先后顺序是

```
AAAA
CCCC
EEEE
IIII
HHHH
BBBB
DDDD
```

答案解析：这道题考察重点是 js 异步执行 宏任务 微任务。

一开始代码执行，输出AAAA. 1

第二行代码开启一个计时器t1(一个称呼)，这是一个异步任务且是宏任务，需要等到1秒后提交。

第四行是个while语句，需要等待3秒后才能执行下面的代码.这里有个问题，就是3秒后上一个计时器t1的提交时间已经过了，但是线程上的任务还没有执行结束，所以暂时不能打印结果，所以它排在宏任务的最前面了。

第五行又输出cccc

第六行又开启一个计时器t2（称呼），它提交的时间是0秒（其实每个浏览器器有默认最小时间的，暂时忽略），但是之前的t1任务还没有执行，还在等待，所以t2就排在t1的后面。（t2排在t1后面的原因是while造成的）都还需要等待，因为线程上的任务还没执行完毕。

第七行new Promise将执行promise函数，它参数是一个回调函数，这个回调函数内的代码是同步的，它的异步核心在于resolve和reject，同时这个异步任务在任务队列中属于微任务，是优先于宏任务执行的，（不管宏任务有多急，反正我是VIP）。所以先直接打印输出同步代码EEEE。第九行中的代码是个不存在的对象，这个错误要抛给reject这个状态，也就是catch去处理，但是它是异步的且是微任务，只有等到线程上的任务执行完毕，立马执行它，不管宏任

务（计时器，ajax等）等待多久了。

第十四行，这是线程上的最后一个任务，打印输出 I III I

我们先找出线程上的同步代码，将结果依次排列出来：AAAA CCCC EEEE IIII

然后我们再找出所有异步任务中的微任务 把结果打印出来 HHHH

最后我们再找出异步中的所有宏任务，这里t1排在前面t2排在后面（这个原因是while造成的），输出结果顺序是 BBBB DDDD

所以综上 结果是 AAAA CCCC EEEE IIII HHHH BBBB DDDD